

Matrix factorisation for recommendations

A rating is a user's taste vector times a movie's feature vector: fit both on the known ratings, then read off the rest.

THE QUESTION

A user has rated a handful of movies out of thousands. How do we predict the rest from everyone's ratings together?

THE INTUITION

A few hidden features, such as how Bollywood or how sci-fi a movie is, explain most ratings. Learn a taste vector per user and a feature vector per movie whose dot products match the known ratings; the same vectors fill the gaps.

KEY TAKEAWAY

Fit $A \approx WH$ on the observed cells only. With one factor fixed the other is least squares: alternate (ALS) or take gradient steps. Too many factors fit the known ratings exactly and guess the rest.

ratings with gaps $\rightarrow a_{ij} \approx w_i \cdot h_j \rightarrow$ loss on known cells $\rightarrow$ ALS or gradient descent $\rightarrow$ predict the ? cells

1 Most cells of the ratings matrix are empty

	Sholay	Swades	Batman	Interstellar	Shawshank
User A	5	4	2	3	2
User B	?	5	1	4	?
User C	4	?	1	5	?

rated: (i, j) in Ω , 11 cells

to predict: 4 cells

$A \in \mathbb{R}^{n \times m}$ holds user i 's rating $a_{ij} \in \{1, \dots, 5\}$ of movie j ; Ω is the set of observed (i, j) . At streaming scale, 2×10^8 users and 15,000 movies make 3×10^{12} cells, and 100 ratings per user fill 0.67% of them. The notebook's class survey (quoted) leaves 204 of 83×10 cells empty.

2 A rating is taste times movie features

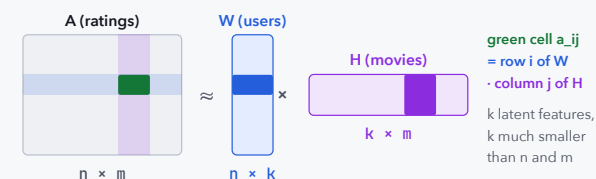
	user A's taste	Sholay h_j	product
Bollywood	3.8	0.95	3.61
Sci-fi	0.8	0.10	0.08
Drama	1.5	0.85	1.28
sum			4.97

user A rated 5

$$\hat{a}_{ij} = w_i^T h_j = \sum_{k=1}^K w_{ik} h_{kj}$$

A movie's features say how Bollywood, sci-fi or dramatic it is; a user's weights say how much each matters. With the lecture's hand-set numbers, user A and Sholay give $4.97 \approx 5$. Real systems learn such features from the ratings, so they are latent and unnamed.

3 Stack the vectors: $A \approx WH$



Row i of $W \in \mathbb{R}^{n \times k}$ is user i 's taste; column j of $H \in \mathbb{R}^{k \times m}$ is movie j 's features. The slides also write N , M and rank r for n , m and k .

4 The lecture's hand-set factors miss some ratings

	Sholay	Swades	Batman	Interstellar	Shawshank
User A	4.96 rated 5	5.31 rated 4	1.28 rated 2	2.00 rated 3	1.73 rated 2
User B	4.84 not rated	5.50 rated 5	4.33 rated 1	6.22 rated 4	3.75 not rated
User C	4.54 rated 4	5.25 not rated	4.59 rated 1	6.15 rated 5	2.83 not rated

off by < 0.5

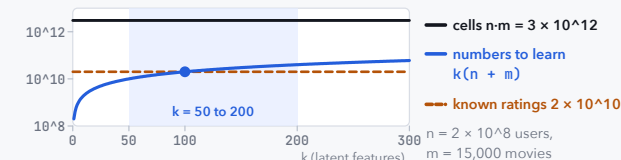
0.5 to 1.5

> 1.5

RMSE on the 11 ratings: 1.76

Users B and C rated Batman 1, yet WH gives 4.33 and 4.59, and user B's Interstellar comes out 6.23, off the scale. So we learn W and H from the ratings instead of guessing them.

5 $k(n + m)$ numbers instead of nm



At $k = 100$: 2.0×10^{10} numbers, 150 times fewer than the cells but as many as the known ratings, so $k = 50$ to 200 needs regularisation (panel 11). Small matrices gain nothing: for 3×5 , $k = 2$ already needs 16 numbers for 15 cells.

6 Fit only the observed cells

	Sho.	Sw.	Bat.	Int.	Shaw.	
User A	0.74	-0.28	0.77	-0.77	0.00	residual $a_{ij} - w_i \cdot h_j$ of the rank-1 fit
User B	skip	0.25	-0.36	-0.17	skip	loss = sum of the 11 squares = 3.36
User C	-0.67	skip	-0.34	0.87	skip	unrated cells: mask 0, no term in the loss

$$\min_{W, H} \sum_{(i, j) \in \Omega} (a_{ij} - w_i^T h_j)^2 = \|P_{\Omega}(A - WH)\|_F^2$$

P_{Ω} keeps observed entries and zeroes the rest. In the notebook the plain norm is nan until the loss is indexed with $\sim \text{isnan}(A)$. The loss is bilinear, so it is not convex in (W, H) jointly; with H fixed it is least squares in W , and vice versa. It has no unique minimiser, since $WH = (WD)(D^{-1}H)$.

Alternate least squares, or step both factors downhill

Fix one factor and every row or column of the other is a small regression; or move both by gradient descent.

7 Fix H : each user's taste is a least-squares fit

	Bolly.	Sci-fi	Drama	W_1	rated	fit	
Sholay	0.95	0.10	0.85		5	4.29	5 equations, 3 unknowns: least squares
Swades	1.00	0.20	0.90	2.78	4	4.68	
Batman	0.05	0.80	0.30	1.66	2	1.99	lecture's guess (3.8, 0.8, 1.5)
Interstellar	0.05	0.95	0.70	1.75	3	2.93	SSE 3.31; fit SSE 0.98
Shawshank	0.05	0.15	0.95		2	2.05	

H^T (fixed) solved

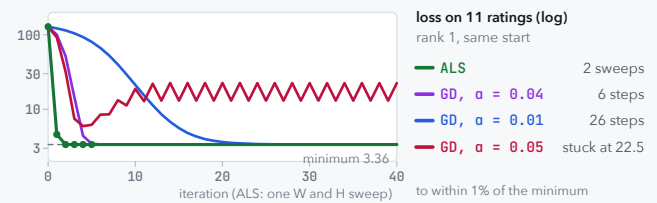
$A[i, :]^T \approx H^T W[i, :]^T$ is $y \approx X\theta$ with X the rows of H^T for the movies user i rated; the normal equation solves it (see the linear regression sheets). With the lecture's H fixed, user A's 5 ratings give (2.78, 1.66, 1.75): squared error 0.98, against 3.31 for the hand-set (3.8, 0.8, 1.5). Users B and C have 3 ratings for 3 unknowns: an exact fit, negative tastes, and Shawshank at 7.37 and 9.75.

8 Fix W : each movie is a fit too; alternate

$H[:, j] \leftarrow \text{LS}(W_o, a_{o,j})$ over the users o who rated movie j . A sweep does every column, then every row. Each solve is exact, so the loss never rises, yet ALS can stop at a stationary point that is not the global minimum. Rows are independent, so they run in parallel; a sweep solves $n + m$ small $k \times k$ systems, more work than one gradient step. R holds λI from the penalty $\lambda(\|W\|_F^2 + \|H\|_F^2)$ that the lecture adds to the loss, which makes each solve a ridge regression (the next sheet). Rank 1, $\lambda = 0$, on the 3×5 matrix:

```
M = ~np.isnan(A)           # observed
R = lam * np.eye(k)        # 0: plain LS
def ls(X, y):               # normal eq.
    G = X.T @ X + R
    return np.linalg.solve(G, X.T @ y)
for t in range(50):        # W, H random
    for j in range(m):     # fix W
        o = M[:, j]
        H[:, j] = ls(W[o], A[o, j])
    for i in range(n):     # fix H
        o = M[i]
        W[i] = ls(H[:, o].T, A[i, o])
print((W @ H)[1, 0].round(2)) # 4.72
```

9 Gradient descent moves W and H together



With $E = P_\Omega(A - WH)$: $\nabla_W L = -2EH^T$ and $\nabla_H L = -2W^T E$, both applied at once. SGD uses one rating: $e_{ij} = a_{ij} - w_i^T h_j$, then $w_i \leftarrow w_i + 2\alpha e_{ij} h_j$ and $h_j \leftarrow h_j + 2\alpha e_{ij} w_i$. The notebook keeps the factors in nn.Embedding tables and trains them with Adam on the masked loss.

10 Read the predictions off WH

	Sholay	Swades	Batman	Interst.	Shawsh.	
User A	5	4	2	3	2	predicted, top: rank 1, $\lambda = 0$ (loss 3.36)
User B	4.72 3.76	5	1	4	2.22 1.35	
User C	4	4.69 4.92	1	5	2.19 1.39	bottom: rank 2, $\lambda = 0.5$ (squared error 0.78)

4 random starts agree

Both fits expect users B and C to like the Bollywood film they skipped more than Shawshank. Rank 1 is one pattern, $h = (8.09, 8.13, 2.33, 7.15, 3.80)$ up to scale: users differ only in how high they rate.

11 More numbers than ratings: exact fit, arbitrary gaps

lecture's start	random start 1	random start 2
5 3.92 2	5 3.60 2	5 3.79 2
4 4 2.21	4 4 2.86	4 4 1.50
16.56 5 1	11.22 5 1	3.19 5 1

The lecture's 3×3 example ($k = 2$) gives each user 2 ratings for 2 unknowns, so one W update ($w_1 = (4.59, 1.35)$, as on the slide) fits all six and the gaps follow the start. Rank 2 on the 3×5 matrix put user B's Sholay at 5.77, 3.57, 1182 and 1096 from four starts. With the λ penalty (panel 8) at 0.5 the starts agree; $\lambda = 1$ underfits (error on known ratings 0.78 up to 2.72).

12 Pause and think

Does rank 1 mean identical tastes?
No. There is one pattern, and each user scales it.

Why add biases? $\hat{a}_{ij} = \mu + b_i + b_j + w_i^T h_j$ absorbs the mean, generous users, liked movies.

Does ALS find the global minimum? No: two rank-2 starts stalled at losses 0.86 and 1.30.
Only clicks, no stars? Implicit feedback: weight each cell by a confidence $c_{ij} = 1 + \alpha \cdot \text{freq}_{ij}$.

A new user has no ratings? Cold start: w_i has no equations; use content or popular items at first.
ALS or SGD at scale? ALS for batch jobs on a cluster, SGD for online updates as ratings arrive.

$a_{ij} \approx w_i^T h_j$ · loss on observed cells only · fix one factor, least squares for the other · few factors, or add λ

Before: the gradient descent sheet (least squares: the linear regression sheets); next: the ridge regression sheet. Source: lecture slides "Matrix factorization for movie recommendation systems" and the movie-recommendation notebook (nipunbatra.github.io/ml-teaching). The matrices and the hand-set W , H are the lecture's; fits, random starts and gradient runs are ours (fixed seeds); survey counts are quoted.